



# **PRESIDENCY UNIVERSITY**

**Department of Computer Science & Engineering**

**COURSE NAME:** Object Oriented Programming

**COURSE CODE:** CSE205

**PROJECT TITLE:** E-Commerce Management System Using Java and SQLite

**Submitted By:**

**Name:** Fahad Fardin Younus

**Student ID:** 253174038

**Submitted To:**

Mostofa Kamal Sagor

Course Instructor

Department of Computer Science

Presidency University

**SUBMISSION DATE:** 7<sup>th</sup> August 2026

## **DECLARATION**

I hereby declare that this project entitled "E-Commerce Management System Using Java and SQLite" is my own original work completed as part of the requirements of the course Object Oriented Programming (CSE205) at Presidency University.

The project has been developed independently using Java, Java Swing, SQLite, and other supporting technologies. All external resources, documentation, and references used during the development process have been properly acknowledged in the reference section. No unauthorized source code has been copied from any online repository or submitted by another student.

I understand that plagiarism or any form of academic misconduct may result in disciplinary action according to the university's academic regulations.

Signed,

Fahad Fardin Younus

253174038

---

# Table of Contents

|                                                 |    |
|-------------------------------------------------|----|
| Chapter 1: Introduction                         | 1  |
| Chapter 2: Literature Review / Background       | 4  |
| Chapter 3: System Requirements                  | 7  |
| Chapter 4: System Design                        | 11 |
| Chapter 5: OOP Concepts Implementation          | 17 |
| Chapter 6: Implementation & Key Code Highlights | 23 |
| Chapter 7 Testing & Validation                  | 28 |
| Chapter 8 Results & Discussions                 | 31 |
| Chapter 9 Conclusions & Future Work             | 34 |
| References                                      | 36 |
| Appendices                                      | 37 |

## List of Figures

|                                         |    |
|-----------------------------------------|----|
| Fig4.1 Architecture                     | 11 |
| Fig4.2 UML Class Diagram                | 12 |
| Fig4.3 Inheritance & Interface Diagram  | 15 |
| Fig4.4 ER Diagram                       | 16 |
| Fig 5.11 User class field encapsulation | 17 |
| Fig 5.12 Product Stock Management       | 17 |
| Fig 5.13 User Class                     | 18 |
| Fig 5.14 Customer class extending user  | 18 |
| Fig 5.15 Admin class extending user     | 18 |
| Fig 5.16 CashPyment Class               | 18 |
| Fig 5.17 CreditCardPay Class            | 19 |
| Fig 5.18 Mobilepayment Class            | 19 |
| Fig 5.19 ProductService Class           | 19 |
| Fig 5.191 Abstract User Class           | 20 |
| Fig 5.192 Payable Class                 | 20 |
| Fig 5.193 ProductService                | 21 |
| Fig 5.194 CartService                   | 21 |
| Fig 5.195 StorefrontPane                | 21 |
| Fig 5.196 Invalid quantity exception    | 22 |
| Fig 5.197 PreparedStatement             | 22 |
| Fig 5.198 ImageUtil                     | 22 |
| fig 6.1 SHA-256                         | 23 |
| fig 6.2 filtering                       | 24 |
| fig 6.3 All UI Designs                  | 26 |
| fig 6.4 Database                        | 26 |

## List of Tables

|                                    |    |
|------------------------------------|----|
| Table 1 Key OOP Concepts           | 5  |
| Table 2 Functional Requirements    | 7  |
| Table 3 Class Descriptions         | 12 |
| Table 4 Database                   | 15 |
| Table 5 Package Structure          | 16 |
| Table 6 Development Environment    | 23 |
| Table 7 Test Class                 | 28 |
| Table 8 Bugs                       | 30 |
| Table 9 Requirement Fulfill Metrix | 31 |
| Table 10 OOP Concepts Coverage     | 32 |
| Table 11 Team Contribution         | 37 |
| Table 12 Development Log           | 37 |

# CHAPTER 1: INTRODUCTION

## 1.1 Background

Today, a lot of buying and selling is handled through digital systems rather than only through physical stores. Businesses need to keep track of products, customers, orders and payments, and doing all of this manually can quickly become difficult. An e-commerce system brings these tasks together and also gives customers a convenient way to browse and order products.

For a small business, using separate records for stock, customers and orders can easily become confusing. The same information may be entered more than once, stock figures can become outdated, and finding a particular order may take unnecessary time. I therefore wanted the project to keep the main activities in one application instead of spreading them across different tools.

For this project, I developed an E-Commerce Management System using Java, Java Swing, and SQLite. The application has separate access for administrators and customers. Customers can create accounts, browse products, use a shopping cart and wishlist, place orders, and check their order history. Administrators can manage products, customers, inventory, orders, and reports through a dedicated dashboard.

Another aim of the project was to use the OOP concepts I learned in class in an actual working program. Instead of making separate examples only to show inheritance or polymorphism, I tried to use those concepts where they made sense in the system. SQLite is used so that the data is still available after the program is closed and opened again.

## 1.2 Problem Statement

Small businesses often need a simple and affordable way to manage products, customer accounts, carts, and orders. When these tasks are handled manually, inventory can become inaccurate, customer information may be duplicated, and orders can be harder to process and track.

Commercial e-commerce platforms already provide these kinds of features, but recreating a system of that scale would not be realistic for this project. My focus was on the core features that could be implemented properly in a desktop application. I also wanted the OOP concepts to be part of the program's design rather than added only as small demonstration examples.

This project aims to fill that gap by building a desktop E-Commerce Management System that combines useful business features with a clear OOP structure. It includes authentication, role-based access, inventory and order management, reporting, persistent database storage, and a graphical user interface in one application

## 1.3 Project Objective

The objectives of this project are:

1. To develop a desktop-based E-Commerce Management System using Java Swing and SQLite.
2. To implement secure user authentication with separate administrator and customer roles.
3. To provide complete product management including adding, updating, deleting, searching, categorizing, and displaying products with images.
4. To implement shopping cart, wish list, checkout, payment, and order management functionalities.
5. To demonstrate core Object-Oriented Programming concepts including encapsulation, inheritance, abstraction, polymorphism, collections, exception handling, and modular package organization.
6. To provide persistent storage using SQLite so that all business information remains available after the application restarts.

## 1.4 Scope

The developed system focuses on managing the core operations of a desktop-based e-commerce application.

### Included Scope

- Customer registration and login
- Administrator login
- Secure password hashing
- Role-based authorization
- Product management (CRUD)
- Product categories
- Product image management
- Shopping cart management
- Wish list functionality
- Checkout process
- Payment processing
- Order placement
- Order history
- Customer management
- Sales reporting and analytics
- SQLite database persistence
- Java Swing graphical user interface

## **Excluded Scope**

The following features are beyond the scope of this project:

- Online payment gateway integration
- Email or SMS notifications
- Multi-vendor marketplace support
- Mobile application
- Cloud database hosting
- Real-time shipment tracking
- Online product reviews and ratings
- Barcode or QR code inventory management

## **1.5 Report Structure**

The report is divided into nine chapters. Chapter 1 introduces the project, its objectives, scope, and motivation. Chapter 2 gives the OOP background and briefly looks at existing e-commerce systems. Chapter 3 explains the system requirements, stakeholders, and use cases. Chapter 4 covers the system design, including the architecture, UML diagrams, package structure, and database design. Chapter 5 shows how the main OOP concepts are used in the actual code. Chapter 6 discusses the development environment, implementation, interface, and data persistence. Chapter 7 describes the testing process, including test cases, edge cases, and bugs that were fixed. Chapter 8 evaluates the final system and its limitations. Chapter 9 concludes the report with a reflection on the project and possible future improvements.



## CHAPTER 2: LITERATURE REVIEW / BACKGROUND

### 2.1 Overview of Object-Oriented Programming

Object-Oriented Programming (OOP) is a way of designing software around objects that contain both data and the methods that work with that data. Compared with a purely procedural approach, OOP can make a larger program easier to organize, reuse, maintain, and extend.

Java was a suitable language for this project because it provides direct support for important OOP features such as encapsulation, inheritance, polymorphism, abstraction, interfaces, exception handling, and collections. It also includes mature tools for building desktop applications.

**The four fundamental OOP principles are:**

**Encapsulation:** Keeps an object's data protected by using private fields and controlled access through methods such as getters and setters. In this project, classes such as User, Product, and Order use encapsulation to keep their data controlled and consistent.

**Inheritance:** Allows one class to reuse common data and behaviour from another class. Here, Customer and Admin share the common features defined in the User class instead of repeating the same code.

**Polymorphism:** Allows the same method or interface to behave differently depending on the object being used. The payment module demonstrates this through different implementations of `processPayment()`, while service methods also use overloading where appropriate.

**Abstraction:** Keeps unnecessary implementation details hidden and exposes only the behaviour that other parts of the program need. Abstract classes and interfaces are used in this project to keep the design cleaner and easier to extend.

In this project, these concepts mainly helped me avoid putting everything into a few large classes. They gave the program a structure where different parts have clearer responsibilities and can be changed with less effect on unrelated code.

### 2.2 Similar Existing Systems

Before deciding on the final features, I looked at a few e-commerce platforms that I was already familiar with. I was not trying to reproduce them completely; I mainly used them to identify common shopping and management workflows that would make sense in my project.

**Amazon** demonstrates how a large e-commerce platform separates customer-facing shopping features from administrative tasks such as product, inventory, order, and payment management. This separation helped guide the overall structure of the project.

**Daraz Bangladesh** uses familiar shopping features such as product browsing, carts, wishlists, categories, customer accounts, and order tracking. Similar workflows were adapted for this desktop application on a much smaller scale.

**Shopify** focuses strongly on the management side of e-commerce, including products, inventory, customers, and sales information. This influenced the admin module of the project, although OmniCommerce remains a local desktop application rather than a full online platform.

## 2.3 Why Java?

I chose Java because the main purpose of the project was to apply OOP, and Java gave me a direct way to work with classes, inheritance, interfaces and exception handling. For the interface I used Swing, since it is already available with Java, and JDBC was used to connect the application to SQLite.

SQLite was selected because it is lightweight, serverless, and simple to bundle with a desktop application. It provides relational data storage without requiring a separate database server to be installed or configured.

This combination also kept the setup manageable for a university project. The interface and database both run locally, so I could focus on the application logic without setting up a web server or a separate database server.

## 2.4 Key OOP Concepts Relevant to the Project

I tried to use each OOP concept for a reason rather than simply including it to meet a requirement. For example, inheritance is useful for the two user roles, while polymorphism fits the different payment classes. The following table shows where each concept appears in the project.

| OOP Concept | Application in the System | Related System Requirement |
|-------------|---------------------------|----------------------------|
|-------------|---------------------------|----------------------------|

|                           |                                                                                                                                              |                                                                                        |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Encapsulation</b>      | Classes such as User, Product, and Order store their data as private attributes with controlled access through getter and setter methods.    | Protect user information, maintain data integrity, and support secure data management. |
| <b>Inheritance</b>        | The Admin and Customer classes inherit common properties and behaviors from the User class.                                                  | Support role-based authentication while reducing code duplication.                     |
| <b>Polymorphism</b>       | Different payment methods implement common payment behavior, and overloaded service methods provide flexible operations.                     | Allow multiple payment options and improve system flexibility.                         |
| <b>Abstraction</b>        | Abstract classes and interfaces define common behavior while hiding implementation details from the user interface.                          | Simplify system design and make future feature expansion easier.                       |
| <b>Exception Handling</b> | Custom exceptions such as DuplicateUsernameException and InvalidQuantityException validate user operations and prevent invalid transactions. | Improve system reliability by handling invalid inputs gracefully.                      |

## **Chapter 3: System Requirements**

### **3.1 Stakeholder Analysis**

The system mainly serves two types of users: administrators and customers. Their responsibilities are different, so the application gives each role access only to the features it needs.

#### **Administrator**

Administrators are responsible for managing the main business operations. Their access includes:

- Add, edit, and delete products
- Upload product images
- Manage customer accounts
- Monitor and update orders
- View sales reports
- Maintain inventory

These functions are available only after an administrator has been authenticated, which prevents normal customer accounts from accessing management features.

#### **Customer**

Customers use the storefront and purchasing features of the application. They can:

- Register and log in
- Browse, search, and filter products
- View product details
- Manage shopping cart and wishlist
- Complete checkout
- View order history
- Update profile information

Customers cannot access administrator functions.

### **3.2 Functional Requirements**

The following functional requirements define the main tasks the completed system is expected to perform.

| ID    | Requirement                                                                                                                                               |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-01 | The system shall allow new customers to register using their personal information.                                                                        |
| FR-02 | The system shall authenticate users using a username and hashed password.                                                                                 |
| FR-03 | The system shall provide separate access privileges for administrators and customers.                                                                     |
| FR-04 | Customers shall browse, search, and filter products by category without logging in.                                                                       |
| FR-05 | Customers shall view detailed product information, including images, descriptions, prices, and stock availability.                                        |
| FR-06 | Customers shall add, update, and remove products from the shopping cart.                                                                                  |
| FR-07 | Customers shall add or remove products from their wishlist for future purchases.                                                                          |
| FR-08 | Customers shall complete the checkout process by selecting a payment method and confirming the order                                                      |
| FR-09 | Customers shall view their previous orders and current order status.                                                                                      |
| FR-10 | Customers shall update their personal profile information, including email, phone number, and address.                                                    |
| FR-11 | Administrators shall perform Create, Read, Update, and Delete (CRUD) operations on products, including uploading product images and assigning categories. |
| FR-12 | Administrators shall view and manage customer accounts.                                                                                                   |
| FR-13 | Administrators shall monitor customer orders and update their processing status.                                                                          |
| FR-14 | The system shall support product image management and display images on the storefront.                                                                   |
| FR-15 | The system shall generate administrative reports, including total sales, orders, customers, and revenue statistics.                                       |
| FR-16 | The system shall store all application data permanently using an SQLite database.                                                                         |
| FR-17 | The system shall securely store user passwords using the SHA-256 hashing algorithm.                                                                       |
| FR-18 | The system shall validate all user inputs before performing database operations.                                                                          |

### 3.3 Non-Functional Requirements

**Performance:** Everyday actions such as logging in, searching for products, and moving between screens should respond quickly enough for normal desktop use.

**Security:** Passwords are stored as SHA-256 hashes, and administrator-only features are restricted through role-based authorization.

**Usability:** The interface uses a consistent layout and controls so that both customers and administrators can navigate the system without unnecessary complexity.

Maintainability: The code is separated into model, service, utility, exception, and user-interface packages so that responsibilities are easier to understand and update.

### **3.4 Use Case Descriptions**

#### **Use Case 1 Customer Registration**

Actor: Customer

Precondition: User is not registered.

Main Flow:

1. Open the registration page.
2. Enter required information.
3. Submit the form.
4. System validates the data.
5. Password is hashed.
6. Customer account is created.

Postcondition: Customer account is successfully stored.

#### **Use Case 2 Customer Purchase**

Actor: Customer

Precondition: Customer is logged in.

Main Flow:

1. Browse products.
2. Add products to cart.
3. Proceed to checkout.
4. Select a payment method.
5. Confirm the order.
6. System updates inventory and stores the order.

Postcondition: Order is successfully placed.

#### **Use Case 3 Product Management**

Actor: Administrator

Precondition: Administrator is logged in.

Main Flow:

1. Open Product Management.
2. Add, edit, or delete a product.
3. Save changes.
4. System validates and updates the database.

Postcondition: Product information is updated.

#### **Use Case 4 Order Management**

Actor: Administrator

Precondition: Administrator is authenticated.

Main Flow:

1. View customer orders.
2. Update order status.
3. Save changes.

Postcondition: Order status is updated.

#### **Use Case 5 Report Generation**

Actor: Administrator

Precondition: Administrator is logged in.

Main Flow:

1. Open Reports.
2. View sales statistics and customer information.

Postcondition: Business reports are displayed.

## Chapter 4: System Design

### 4.1 System Architecture

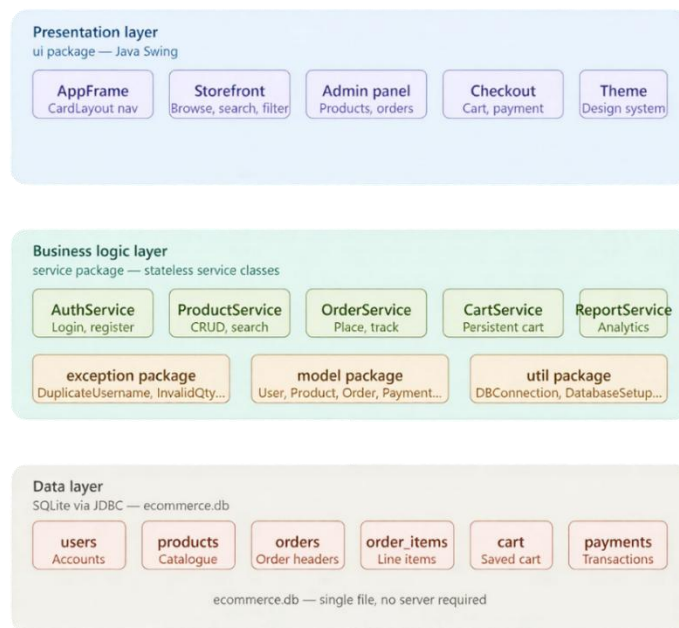


Fig4.1 Architecture

**Presentation Layer (ui package):** All Java Swing components including AppFrame (the main window using CardLayout for screen navigation), panels (LoginPanel, RegisterPanel, StorefrontPanel, AdminDashboardPanel, CustomerDashboardPanel), and frames (CartFrame, CheckoutFrame, OrdersFrame, WishlistFrame, ReportsFrame, ProductFormDialog, ProductDetailsDialog, ProfileFrame, CustomerManagementFrame, OrderDetailsFrame). A central Theme class provides all colours and fonts, and UiFactory provides reusable component creation methods.

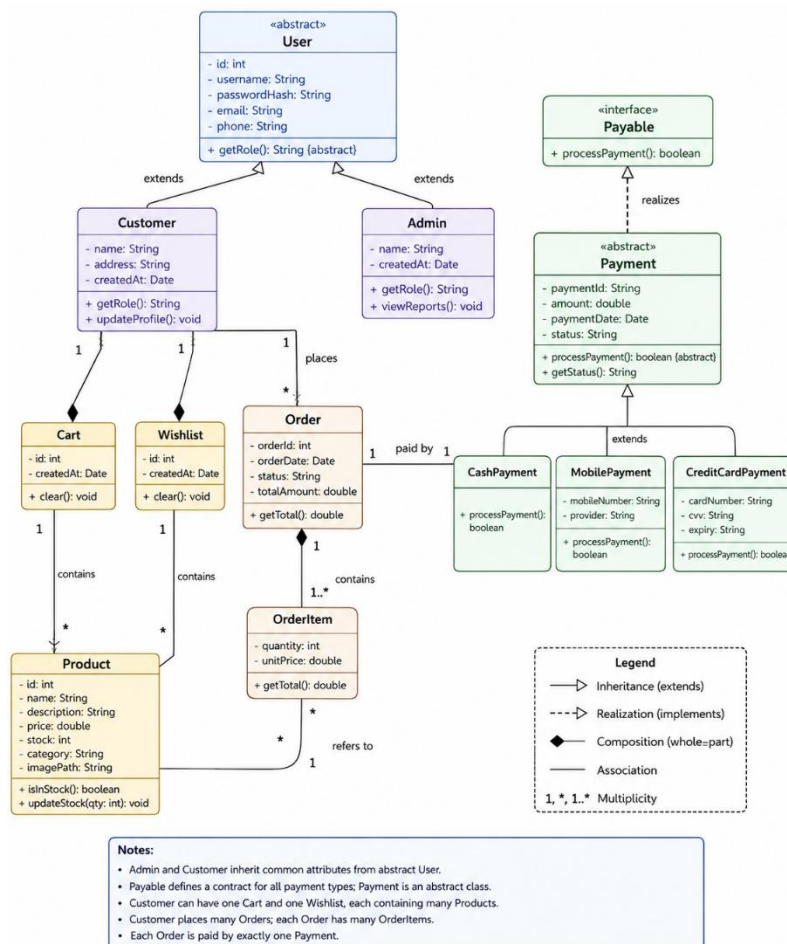
**Business Logic Layer (service package):** The service classes contain the main application logic. AuthService handles login, registration, and password hashing; ProductService manages product CRUD and search; OrderService handles order placement and database transactions; CartService and WishlistService manage saved customer items; ReportService produces summary information; UserService manages customer profiles; and PaymentService records payment results.

I divided OmniCommerce into three main layers. Early on, it would have been easy to place database queries and application logic directly inside the Swing event handlers, but that would make the code harder to follow. Keeping the UI, services and database-related code separate made the project easier for me to organize.



Data Layer (util + SQLite): DBConnection manages the shared database connection, while DatabaseSetup creates the required tables when the program starts for the first time. The ecommerce.db file stores the application's data in seven tables: users, products, orders, order\_items, cart, payments, and wishlist.

## 4.2 UML Class Diagram



The class diagram on the next page shows how the main classes are connected. I included the classes that are most important to the program, such as User, Product, Order, Payment and Cart, together with the relationships between them.

The UML Class Diagram illustrates:

- Class attributes
- Public methods
- Inheritance hierarchy
- Associations
- Dependencies
- Multiplicities

Fig4.2 UML Class Diagram

## 4.3 Class Descriptions

The major classes used in the system are described below.

| Class    | Package | Purpose                     | Key Methods                                     |
|----------|---------|-----------------------------|-------------------------------------------------|
| User     | model   | Abstract base for all users | getRole() [abstract], setPassword(), setEmail() |
| Customer | model   | Registered customer account | getRole() → "CUSTOMER"                          |

|                 |         |                                       |                                                                                                          |
|-----------------|---------|---------------------------------------|----------------------------------------------------------------------------------------------------------|
| Admin           | model   | Administrator account                 | getRole() → "ADMIN"                                                                                      |
| Product         | model   | Store product entity                  | reduceStock(), all getters/setters                                                                       |
| Order           | model   | Customer order record                 | getOrderId(), getStatus(), setStatus()                                                                   |
| OrderItem       | model   | Single item within an order           | getTotal() = quantity × price                                                                            |
| Payment         | model   | Abstract payment base                 | processPayment() [abstract], getPaymentStatus()                                                          |
| CashPayment     | model   | Cash on delivery payment              | processPayment() → always true                                                                           |
| MobilePayment   | model   | bKash/Nagad payment                   | processPayment() with number validation                                                                  |
| CreditCardPay   | model   | Credit card payment                   | processPayment() with card validation                                                                    |
| Cart            | model   | In-memory cart helper                 | addItem(), calculateTotal(), clearCart()                                                                 |
| AuthService     | service | Login, registration, password hashing | login(), register(), addAdmin(), hashPassword()                                                          |
| ProductService  | service | Product CRUD and search               | addProduct(), getAllProducts(), searchProduct() [×2], updateProduct(), deleteProduct(), getProductById() |
| OrderService    | service | Order placement and retrieval         | placeOrder(), placeOrderAndReturnId(), getAllOrders(), getOrdersByCustomer(), updateOrderStatus()        |
| CartService     | service | Persistent cart management            | addToCart(), getCart(), removeFromCart(), clearCart(), checkout(), getCartTotal()                        |
| WishlistService | service | Saved product management              | add(), remove(), contains(), getAll()                                                                    |

|               |               |                                  |                                                                                                           |
|---------------|---------------|----------------------------------|-----------------------------------------------------------------------------------------------------------|
| ReportService | service       | Analytics and reporting          | getTotalSales(), getTotalOrders(),<br>getTotalCustomers(),<br>getLowStockProducts(),<br>getSalesByMonth() |
| DBConnection  | util          | Singleton database connection    | getConnection(),<br>closeConnection()                                                                     |
| DatabaseSetup | util          | Table creation on first launch   | createTables()                                                                                            |
| ImageUtil     | util          | Product image file management    | copyProductImage(), scaledIcon()                                                                          |
| Theme         | ui            | Central design system constants  | Colour and font constants                                                                                 |
| UiFactory     | ui.components | Reusable Swing component factory | primaryButton(),<br>secondaryButton(), textField(),<br>cardPanel()                                        |
| AppFrame      | ui            | Main application window          | showScreen(), completeLogin(),<br>logout(), isAdmin()                                                     |

## 4.4 Inheritance & interface Design

Figure 4.3 shows the inheritance structure used in the project. User is an abstract base class containing the information shared by every user. Customer and Admin extend it, so common fields and methods do not have to be written twice, while each subclass can still provide its own role-specific behaviour.

The payment design uses both an interface and an abstract class. Payable defines the behaviour a payment object must provide, while Payment stores information that is common to all payments. CashPayment, MobilePayment, and CreditCardPay extend Payment and each implement processPayment() in their own way. Because the rest of the program can work with the common payment type, a new payment method can be added later without redesigning the whole checkout flow.

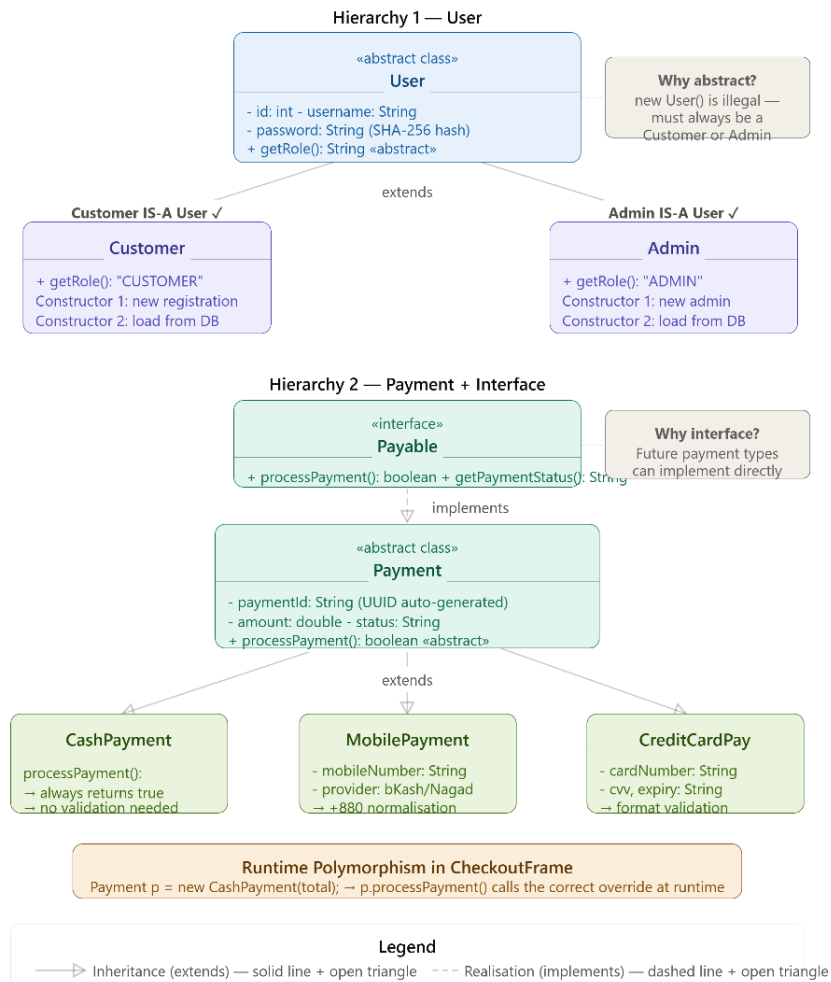


Fig4.3 Inheritance & Interface Diagram

## 4.5 Data Model

The SQLite database contains seven tables:

| Table       | Key Columns                                                         | Purpose                                      |
|-------------|---------------------------------------------------------------------|----------------------------------------------|
| users       | id, username (UNIQUE), password, role, email, phone, address        | Stores all user accounts                     |
| products    | id, name, description, price, stock, category, image_path           | Stores all product catalogue entries         |
| orders      | id, customer_id (FK→users), total_amount, status, order_date        | Stores order header records                  |
| order_items | id, order_id (FK→orders), product_id (FK→products), quantity, price | Stores individual line items per order       |
| cart        | id, customer_id (FK→users), product_id (FK→products), quantity      | Stores persistent cart contents per customer |

|          |                                                                                         |                                        |
|----------|-----------------------------------------------------------------------------------------|----------------------------------------|
| payments | id, order_id (FK→orders), amount, method, status, payment_date                          | Records payment outcome for each order |
| wishlist | id, customer_id (FK→users), product_id (FK→products)<br>UNIQUE(customer_id, product_id) | Stores saved products per customer     |

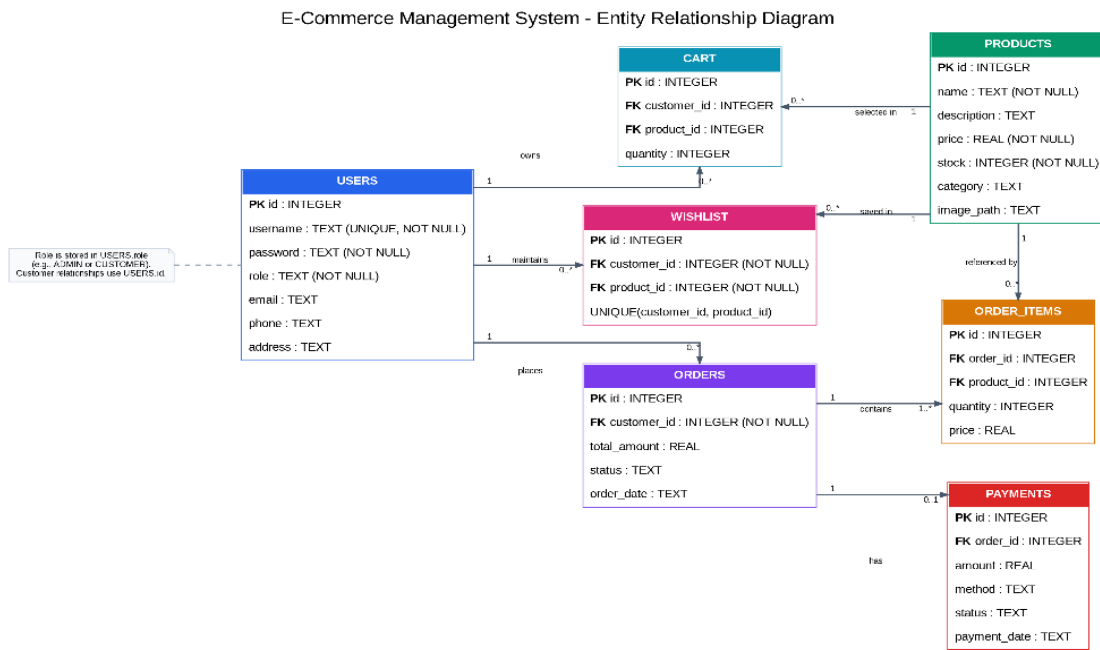


Fig4.4 ER Diagram

## 4.6 Package Structure

The source code is divided into five main packages. This keeps classes with similar responsibilities together and makes the project easier to navigate.

| Package   | Contents                                                                                                            | Responsibility                             |
|-----------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| model     | User, Customer, Admin, Product, Order, OrderItem, Payment, CashPayment, MobilePayment, CreditCardPay, Cart, Payable | Entity classes representing domain objects |
| service   | AuthService, ProductService, OrderService, CartService, WishlistService, ReportService, UserService, PaymentService | Business logic and database operations     |
| ui        | AppFrame, Theme, panels/, frames/, components/                                                                      | All graphical user interface components    |
| exception | DuplicateUsernameException, InvalidQuantityException, PaymentFailedException, ProductNotFoundException              | Custom domain specific exceptions          |
| util      | DBConnection, DatabaseSetup, ImageUtil                                                                              | Shared utility and infrastructure classes  |

## Chapter 5: OOP Concepts Implementation

### 5.1 Encapsulation

Encapsulation is used throughout the model classes by keeping fields private and accessing them through methods. This prevents other classes from changing an object's internal state directly and gives the model class control over how its data is updated.

#### Example 1: User class field encapsulation:

```
public abstract class User {
    private int id;
    private String username;
    private String password;
    private String email;
    private String phone;
    private String address;
    private String role;

    public String getPassword(){
        return password;
    }

    public void setPassword(String password){
        this.password = password;
    }
    public void setEmail(String email){
        this.email = email;
    }
    public void setPhone(String phone){
        this.phone = phone;
    }
}
```

The password field in User is one example. The application stores the hashed value rather than a plain-text password, and the field itself is private. Other classes therefore cannot directly change the value without going through the class methods.

Fig 5.11 User class field encapsulation

#### Example 2 Product stock management:

```
public class Product {
    private int id;
    private String name;
    private String description;
    private double price;
    private int stockQuantity;
    private String category;
    private String imagePath;

    public void reduceStock(int quantity) {
        if (quantity <= stockQuantity) {
            this.stockQuantity -= quantity;
        } else {
            throw new IllegalArgumentException(s: "Not enough stock!");
        }
    }

    public int getStockQuantity() {
        return stockQuantity;
    }
}
```

Product stock is another practical example. Instead of allowing any part of the program to reduce stock however it wants, reduceStock() checks the requested quantity first. This helps prevent the stock value from becoming negative.

Fig 5.12 Product Stock Management

### 5.2 Inheritance

Inheritance is useful when several classes share the same basic data or behaviour. In this project it reduces repetition by placing common user information in one parent class.

```

public abstract class User {
    private int id;
    private String username;
    private String password;
    private String email;
    private String phone;
    private String address;
    private String role;

    public User(int id, String username, String password, String email, String phone, String address, String role){
        this.id = id;
        this.username = username;
        this.password = password;
        this.email = email;
        this.phone = phone;
        this.address = address;
        this.role = role;
    }

    public abstract String getRole();
}

```

User stores the information shared by both account types, including username, password hash, email, phone number, address, and role. Admin and Customer inherit these fields and then add the behaviour needed for their own roles.

Fig 5.13 User Class

```

public class Customer extends User {

    public Customer(String username, String password, String email, String phone, String address) {
        super(id: 0, username, password, email, phone, address, role: "CUSTOMER");
    }

    public Customer(int id, String username, String password, String email, String phone, String address) {
        super(id, username, password, email, phone, address, role: "CUSTOMER");
    }

    @Override
    public String getRole() {
        return "CUSTOMER";
    }
}

```

The constructors use super() to pass the shared values to the User constructor. This keeps initialization of inherited fields in one place instead of repeating it in each subclass.

Fig 5.14 Customer class extending user

```

public class Admin extends User {

    public Admin(String username, String password, String email, String phone, String address) {
        super(id: 0, username, password, email, phone, address, role: "ADMIN");
    }

    public Admin(int id, String username, String password, String email, String phone, String address) {
        super(id, username, password, email, phone, address, role: "ADMIN");
    }

    @Override
    public String getRole() {
        return "ADMIN";
    }
}

```

Fig 5.15 Admin class extending user

## 5.3 Polymorphism (Method Overriding)

Runtime polymorphism appears in the payment system. CashPayment, MobilePayment, and CreditCardPay all override processPayment(), but the actual validation and result depend on which payment object is being used.

```

public class CashPayment extends Payment {
    public CashPayment(double amount) {
        super(amount);
    }

    @Override
    public boolean processPayment() {
        System.out.println(x: "Cash payment processing...");
        setStatus(status: "PAID");
        return true;
    }
}

```

Fig 5.16 CashPayment Class

```

public class CreditCardPay extends Payment{

@Override
public boolean processPayment() {
    // Validate card number - must be 16 digits
    if (cardNumber == null || cardNumber.length() != 16) {
        setStatus(status: "FAILED");
        System.out.println(x: "Invalid card number!");
        return false;
    }

    if (cvv == null || cvv.length() != 3) {
        setStatus(status: "FAILED");
        System.out.println(x: "Invalid CVV!");
        return false;
    }

    if (expiry == null || !expiry.matches(regex: "\\d{2}/\\d{2}")) {
        setStatus(status: "FAILED");
        System.out.println(x: "Invalid expiry date! Use MM/YY format.");
        return false;
    }

    if (Math.random() < 0.9) {
        setStatus(status: "PAID");
        System.out.println(x: "Credit card payment successful!");
        return true;
    } else {
        setStatus(status: "FAILED");
        System.out.println(x: "Card declined!");
        return false;
    }
}
}

```

Fig 5.17 CreditCardPay Class

```

public class MobilePayment extends Payment {

@Override
public boolean processPayment() {
    if (mobileNumber == null) {
        setStatus(status: "FAILED");
        System.out.println(x: "Invalid mobile number!");
        return false;
    }

    // Strip +880 if present and replace with 0
    String normalizedNumber = mobileNumber;
    if (mobileNumber.startsWith(prefix: "+880")) {
        normalizedNumber = "0" + mobileNumber.substring(beginIndex: 4);
    }

    // Now check it's 11 digits starting with 01
    if (normalizedNumber.length() != 11 || !normalizedNumber.startsWith(prefix: "01")) {
        setStatus(status: "FAILED");
        System.out.println(x: "Invalid Bangladesh mobile number!");
        return false;
    }

    // Validate provider
    if (!provider.equals(anObject: "bKash") && !provider.equals(anObject: "Nagad")) {
        setStatus(status: "FAILED");
        System.out.println(x: "Invalid provider! Use bKash or Nagad.");
        return false;
    }
}
}

```

Fig 5.18 Mobilepayment Class

## 5.4 Polymorphism (Method overloading)

ProductService also uses method overloading. Methods such as addProduct() and searchProduct() have more than one parameter list, allowing related operations to use the same method name in different situations.

```

public class ProductService {
    public static void addProduct(String name, String description, double price, int stock, String category) {
        addProduct(name, description, price, stock, category, imagePath: null);
    }

    public static void addProduct(String name, String description, double price, int stock, String category, String imagePath) {...}

    public static List<Product> getAllProducts() {...}

    public static List<Product> searchProduct(String name) {
        return findProducts(sql: "SELECT * FROM products WHERE name LIKE ? ORDER BY id DESC", "%" + name + "%", second: null);
    }

    public static List<Product> searchProduct(String name, String category) {
        return findProducts(
            sql: "SELECT * FROM products WHERE name LIKE ? AND category LIKE ? ORDER BY id DESC", "%" + name + "%", "%" + category + "%");
    }

    private static List<Product> findProducts(String sql, String first, String second) {
        List<Product> products = new ArrayList<>();
        try {...}
    } catch (SQLException e) {...}
    return products;
}

public static void updateProduct(int id, String name, String description, double price, int stock, String category) {...}

public static void updateProduct(int id, String name, String description, double price, int stock, String category, String imagePath) {...}

```

Fig 5.19 ProductService Class

This makes the service easier to read because closely related operations keep a consistent name while still accepting different inputs.



## 5.5 Abstraction (Abstract Class)

User is declared as an abstract class because a generic user has no meaningful role in this system. The program should create either a Customer or an Admin, not a plain User object.

```
public abstract class User {
    private int id;
    private String username;
    private String password;
    private String email;
    private String phone;
    private String address;
    private String role;

    public User(int id, String username, String password, String email, String phone, String address, String role){
        this.id = id;
        this.username = username;
        this.password = password;
        this.email = email;
        this.phone = phone;
        this.address = address;
        this.role = role;
    }

    public abstract String getRole();
}
```

Fig 5.191 Abstract User Class

Trying to create new User() would therefore cause a compile-time error. This design also makes the role structure clearer because every real user object must belong to one of the supported account types.

## 5.6 Abstraction (Interface)

The Payable interface defines a contract that any payable entity must implement:

Payable defines the basic contract required from a payment object. Any class that implements it must provide the payment-processing behaviour expected by the rest of the system.

```
package model;

public interface Payable {
    boolean processPayment();
    String getPaymentStatus();
}
```

Fig 5.192 Payable Class

Payment implements Payable, and all three payment subclasses inherit this implementation. The interface separates what a payment object does (the contract) from how it does it (the implementation). In the future, a new payment type such as CryptoPayment could implement Payable directly without extending Payment, demonstrating the flexibility that interfaces provide over abstract classes alone.

## 5.7 Collections Framework

Java collection classes are used in several parts of OmniCommerce to work with groups of products, cart items, orders, and categories.

```
private static List<Product> findProducts(String sql, String first, String second) {
    List<Product> products = new ArrayList<>();
    try {
        Connection conn = DBConnection.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        if (first != null) stmt.setString(parameterIndex: 1, first);
        if (second != null) stmt.setString(parameterIndex: 2, second);
        ResultSet rs = stmt.executeQuery();
        while (rs.next()) products.add(mapProduct(rs));
        stmt.close();
    } catch (SQLException e) { ...
    }
    return products;
}
```

Fig 5.193 ProductService

```
public static List<OrderItem> getCart(int customerId) {
    List<OrderItem> cartItems = new ArrayList<>();
    try {
        Connection conn = DBConnection.getConnection();
        String sql = "SELECT c.quantity, p.id, p.name, p.description, " + "p.price, p.stock, p.category, "
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setInt(parameterIndex: 1, customerId);
        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            int quantity = rs.getInt(columnLabel: "quantity");
            int id = rs.getInt(columnLabel: "id");
            String name = rs.getString(columnLabel: "name");
            String description = rs.getString(columnLabel: "description");
            double price = rs.getDouble(columnLabel: "price");
            int stock = rs.getInt(columnLabel: "stock");
            String category = rs.getString(columnLabel: "category");
            String imagePath = rs.getString(columnLabel: "image_path");

            Product product = new Product(id, name, description, price, stock, category, imagePath);
            cartItems.add(new OrderItem(product, quantity));
        }
        stmt.close();
    } catch (SQLException e) {
        System.out.println("Cart retrieval error: " + e.getMessage());
    }
    return cartItems;
}
```

Fig 5.194 CartService

```
private void refreshCategories() {
    Object selected = categoryBox.getSelectedItemAt();
    Set<String> categories = new LinkedHashSet<>();
    for (Product product : allProducts) {
        if (product.getCategory() != null && !product.getCategory().isBlank()) {
            categories.add(product.getCategory());
        }
    }
    categoryBox.removeAllItems();
    categoryBox.addItem(item: "All Categories");
    categories.stream().sorted(String.CASE_INSENSITIVE_ORDER).forEach(categoryBox::addItem);
    if (selected != null) categoryBox.setSelectedItem(selected);
    if (categoryBox.getSelectedIndex() < 0) categoryBox.setSelectedIndex(anIndex: 0);
}
```

Fig 5.195 StorefrontPanel

StorefrontPanel uses a LinkedHashSet when building the category filter. A set prevents duplicate category names, while LinkedHashSet also keeps them in insertion order. This was a better fit than a simple list for the dropdown.

## 5.8 Exception Handling

The project includes four custom exception classes for errors that are specific to the application's business rules.

These exceptions can carry useful information in addition to an error message. For example, CheckoutFrame can read the requested and available quantities from

InvalidQuantityException and show the user a more specific explanation of why checkout cannot continue.

```
public class InvalidQuantityException extends Exception{
    private int requestedQuantity;
    private int availableQuantity;

    public int getRequestedQuantity(){...}
    public int getAvailableQuantity(){...}

    public InvalidQuantityException(int requestedQuantity, int availableQuantity){
        super("Invalid quantity! Requested: " + requestedQuantity + " but only " +
            this.availableQuantity + " available");
        this.requestedQuantity = requestedQuantity;
    }
}
```

Fig 5.196 Invalid quantity exception

## 5.9 File I/O / Serialization

SQLite is accessed through JDBC for persistent data storage. Database operations use PreparedStatement with parameters rather than building SQL statements directly from user input, which also reduces the risk of SQL injection.

```
public static User login(String username, String password){

    try {
        Connection conn = DBConnection.getConnection();
        String sql = "SELECT * FROM users WHERE username = ? AND password = ?";
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setString(1, username);
        stmt.setString(2, hashPassword(password));

        ResultSet rs = stmt.executeQuery();
    }
}
```

Fig 5.197 PreparedStatement

The project also performs file I/O when product images are added. ImageUtil copies the selected image into the data/product\_images/ folder, gives it a UUID-based filename, and returns the saved path so it can be stored with the product record.

```
public static String copyProductImage(File source) throws IOException {
    if (source == null || !source.isFile()) return null;
    Files.createDirectories(PRODUCT_IMAGE_DIR);
    String extension = extensionOf(source.getName());
    String filename = UUID.randomUUID() + extension;
    Path target = PRODUCT_IMAGE_DIR.resolve(filename);
    Files.copy(source.toPath(), target, StandardCopyOption.REPLACE_EXISTING);
    return target.toString().replace(oldChar: '\\', newChar: '/');
}
```

Fig 5.198 ImageUtil

## Chapter 6: Implementation & Key Code Highlights

### 6.1 Development Environment

| Component        | Details                                                                                                           |
|------------------|-------------------------------------------------------------------------------------------------------------------|
| IDE              | Visual Studio Code with Java Extension Pack                                                                       |
| JDK Version      | Java 26.0.1 (build 26.0.1+8-34)                                                                                   |
| Database         | SQLite via sqlite-jdbc-3.53.2.0.jar                                                                               |
| GUI Framework    | Java Swing (built into JDK no additional dependency)                                                              |
| Version Control  | Git with GitHub ( <a href="https://github.com/fahady724/eccommerce">https://github.com/fahady724/eccommerce</a> ) |
| Operating System | Windows 11 (64-bit)                                                                                               |
| Build Tool       | No build tool compiled directly with javac                                                                        |

### 6.2 Key Algorithm Implementations

#### SHA-256 Password Hashing

Before a password is saved, AuthService hashes it using Java's MessageDigest class with SHA-256. The database therefore stores the resulting hash instead of the original password text.

```
private static String hashPassword(String password) {
    try {
        java.security.MessageDigest md = java.security.MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest(password.getBytes());
        StringBuilder sb = new StringBuilder();
        for (byte b : hash) {
            sb.append(String.format(format: "%02x", b));
        }
        return sb.toString();
    } catch (Exception e) {
        return password;
    }
}
```

fig 6.1 SHA-256

#### Client-Side Search and Filter Algorithm

StorefrontPanel performs search, category filtering, and sorting on the products already loaded in memory. This gives the user immediate filtering without sending a new database query for every change in the search controls.

```
private void applyFilters() {
    if (productGrid == null) return;
    String query = searchField.getText().trim().toLowerCase(Locale.ROOT);
    String category = categoryBox.getSelectedItem() == null ? "All Categories" : categoryBox.getSelectedItem().toString();

    List<Product> filtered = new ArrayList<>();
    for (Product product : allProducts) {
        String name = product.getName() == null ? "" : product.getName().toLowerCase(Locale.ROOT);
        String description = product.getDescription() == null ? "" : product.getDescription().toLowerCase(Locale.ROOT);
        boolean matchesText = query.isEmpty() || name.contains(query) || description.contains(query);
        boolean matchesCategory = "All Categories".equals(category) || category.equalsIgnoreCase(product.getCategory());
        if (matchesText && matchesCategory) filtered.add(product);
    }

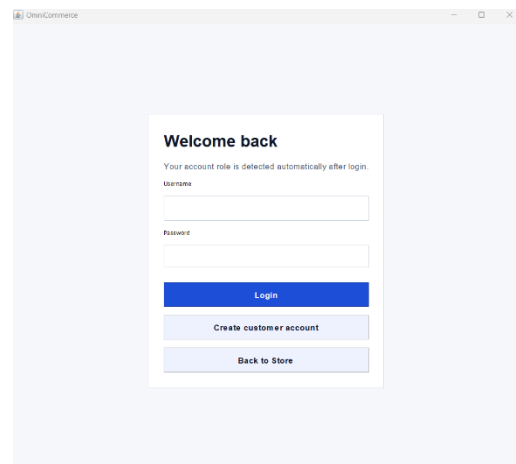
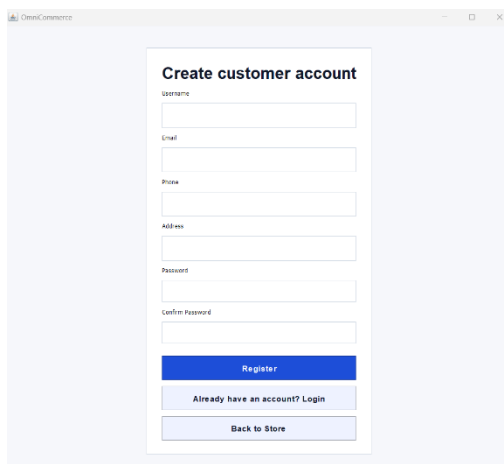
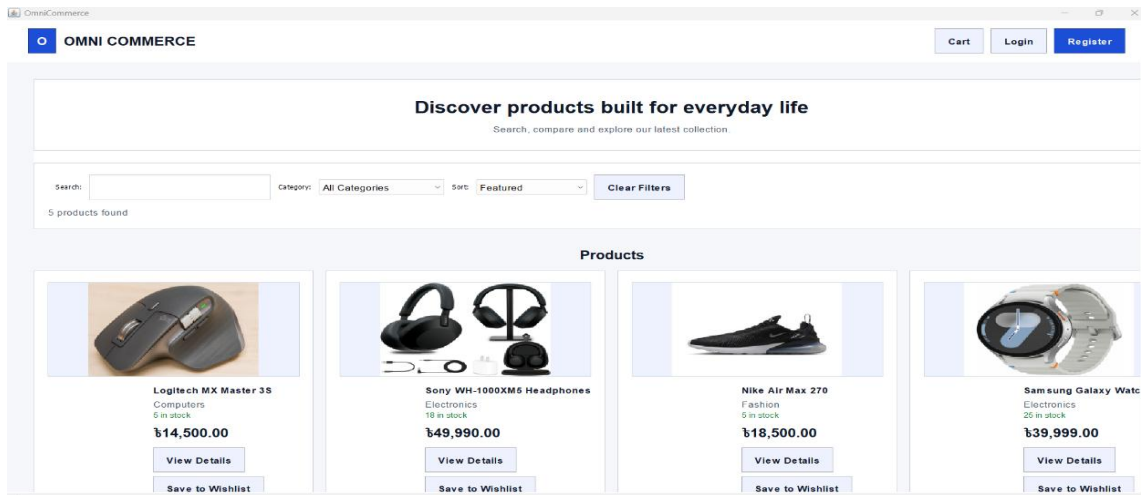
    String sort = sortBox.getSelectedItem() == null ? "Featured" : sortBox.getSelectedItem().toString();
    if ("Price: Low to High".equals(sort)) {
        filtered.sort(Comparator.comparingDouble(Product::getPrice));
    } else if ("Price: High to Low".equals(sort)) {
        filtered.sort(Comparator.comparingDouble(Product::getPrice).reversed());
    } else if ("Name: A to Z".equals(sort)) {
        filtered.sort(Comparator.comparing(Product::getName, String.CASE_INSENSITIVE_ORDER));
    }

    renderProducts(filtered);
}
```

fig 6.2 filtering

## 6.3 User Interface Design

For the interface, I used Swing with CardLayout for the main screens. I preferred switching panels inside AppFrame because opening a separate window for every screen quickly became messy during development. Some supporting screens still use their own frames or dialogs where that is more practical.



OmniCommerce - Shopping Cart

## Shopping Cart

| Product                    | Price      | Quantity | Subtotal   |
|----------------------------|------------|----------|------------|
| Logitech MX Master 3S      | ₺14,500.00 | 2        | ₺29,000.00 |
| Sony WH-1000XM5 Headphones | ₺49,990.00 | 1        | ₺49,990.00 |
| Nike Air Max 270           | ₺18,500.00 | 1        | ₺18,500.00 |

Total: **₺97,490.00**

Remove Selected

Clear Cart

Checkout

OmniCommerce

## Admin Dashboard

Products  
5

Customers  
1

Orders  
1

Revenue  
₺97,490.00

### Product Management

| ID | Name                       | Category             | Price      | Stock |
|----|----------------------------|----------------------|------------|-------|
| 5  | Logitech MX Master 3S      | Computers            | ₺14,500.00 | 3     |
| 4  | Sony WH-1000XM5 Headphones | Electronics          | ₺49,990.00 | 17    |
| 3  | Nike Air Max 270           | Fashion              | ₺18,500.00 | 4     |
| 2  | Samsung Galaxy Watch7      | Electronics          | ₺39,999.00 | 25    |
| 1  | iPhone 13                  | Mobile & Accessories | ₺30,000.00 | 50    |

Refresh

Reports & Analytics

Manage Customers

Manage Orders

Add Product

Edit Selected

Delete Selected

OmniCommerce - Checkout

## Checkout

Review your order and choose a payment method.

### Delivery information

Customer: fahad12

Email: asfghuias@gmail.com

Phone: 01972742013

Address: afdsaf

| Product           | Qty | Price      | Subtotal   |
|-------------------|-----|------------|------------|
| Logitech MX Ma... | 2   | ₺14,500.00 | ₺29,000.00 |
| Sony WH-1000X...  | 1   | ₺49,990.00 | ₺49,990.00 |
| Nike Air Max 270  | 1   | ₺18,500.00 | ₺18,500.00 |

### Payment method

☒ Cash on Delivery

☐ Mobile Banking

☐ Credit Card

Pay when your order is delivered.

Back to Cart

Total: **₺97,490.00**

Place Order

A central Theme class defines all visual constants:

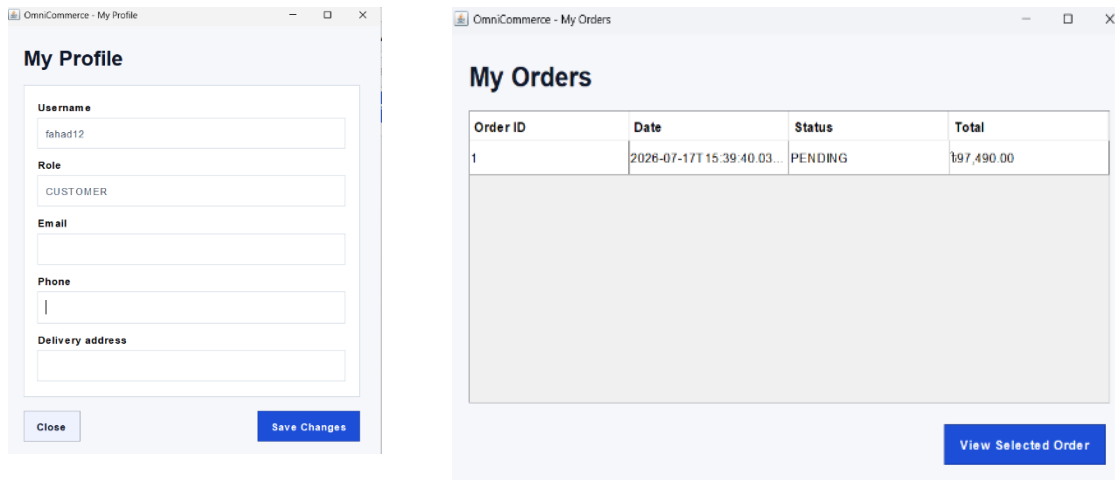
- Background: #F6F7FB (light grey)
- Surface: White with #CBD5E1 border
- Primary action: #1D4ED8 (blue)

- Danger action: #B91C1C (red)

- Success indicator: #15803D (green)

- Typography: SansSerif

- Title 28pt Bold, Heading
- 20pt Bold, Body
- 14pt Regular



The main screens include the Storefront for browsing and filtering products, Login and Register for account access, the Admin Dashboard for product and management tasks, Reports for sales and stock information, Cart for reviewing selected items, and Checkout for confirming delivery details and choosing a payment method.

## 6.4 Data Persistence Implementation

Persistent data is stored in SQLite through JDBC. DBConnection follows the Singleton pattern so the application reuses one shared connection instead of creating a separate connection every time a service needs database access.

```
public class DBConnection {
    private static final String URL = "jdbc:sqlite:ecommerce.db";
    private static Connection connection = null;

    public static Connection getConnection() {
        if (connection == null) {
            try {
                connection = DriverManager.getConnection(URL);
                System.out.println(x: "Database connected successfully!");
            } catch (SQLException e) {
                System.out.println("Connection failed: " + e.getMessage());
            }
        }
        return connection;
    }
}
```

fig 6.3 Database

DatabaseSetup uses CREATE TABLE IF NOT EXISTS when initializing the database, so running the setup again does not recreate existing tables. It also attempts simple schema updates when needed and ignores the expected error when a column is already present, allowing an older database file to continue working after minor changes.

## 6.5 Authentication & Authorization

`AuthService.login()` handles authentication by hashing the entered password and querying the database for a matching username and hash. If a record is found, the stored role determines whether the method returns a Customer or an Admin object.

Authorization is enforced at multiple levels:

- `AppFrame.isAdmin()` checks the current user's role before showing admin-only UI elements.
- `StorefrontPanel` hides "Add to Cart" and wishlist buttons for admin users.
- `CartFrame`, `CheckoutFrame`, and `OrdersFrame` redirect to login if no user is authenticated.
- `AdminDashboardPanel` is reached only after a successful administrator login. Normal navigation does not provide a route to this panel for unauthenticated users or customer accounts.



## Chapter 7 Testing & Validation\

### 7.1 Testing Strategy

Most of my testing was manual because this project does not yet have a JUnit test suite. I went through the features from the interface, checked whether the result matched what I expected, and also inspected the SQLite database when I needed to confirm that data had been saved correctly. I grouped the testing into three stages:

- Unit-level testing: Individual service methods were called with known inputs from Main.java, and I checked the resulting database records using DB Browser for SQLite.
- Integration testing: Complete user journeys were tested from beginning to end, for example registration → login → browsing → adding to cart → checkout → order history.
- Edge-case testing: Invalid and unusual inputs were entered deliberately to confirm that validation and exception handling stopped incorrect operations without crashing the application.

### 7.2 Test Class

| Test ID | Description                    | Input                                     | Expected Output                         | Actual Output                      | Pass/Fail |
|---------|--------------------------------|-------------------------------------------|-----------------------------------------|------------------------------------|-----------|
| TC-01   | Register with valid data       | fahad12, pass12345, email, phone, address | Account created; redirect to login      | Account created; redirect to login | PASS      |
| TC-02   | Register duplicate username    | fahad12 (already exists)                  | Error: Username already exists          | Error dialog shown                 | PASS      |
| TC-03   | Login with correct credentials | fahad12 / pass12345                       | Login success; storefront shown         | Login success; storefront shown    | PASS      |
| TC-04   | Login with wrong password      | fahad12 / wrongpass                       | Error: Invalid username or password     | Error dialog shown                 | PASS      |
| TC-05   | Login as admin                 | fahad / 14529882                          | Admin panel available in navigation     | Admin panel shown                  | PASS      |
| TC-06   | Add product (admin)            | Valid name, price, stock, category        | Product appears in table and storefront | Product added successfully         | PASS      |

|       |                                 |                                          |                                              |                                              |      |
|-------|---------------------------------|------------------------------------------|----------------------------------------------|----------------------------------------------|------|
| TC-07 | Add product with negative price | -500                                     | Validation error: price cannot be negative   | Error dialog shown                           | PASS |
| TC-08 | Search products by name         | "iPhone"                                 | Only iPhone products displayed               | Filtered results shown                       | PASS |
| TC-09 | Add to cart (logged in)         | iPhone 15, qty 1                         | Cart count increments; item in cart          | Cart updated correctly                       | PASS |
| TC-10 | Add to cart (not logged in)     | Click Add to Cart                        | Prompt to login shown                        | Login prompt appeared                        | PASS |
| TC-11 | Checkout with cash              | Items in cart, select Cash               | Order placed; cart cleared                   | Order created successfully                   | PASS |
| TC-12 | Checkout with invalid card      | 15-digit card number                     | Error: Invalid card number                   | Error dialog shown                           | PASS |
| TC-13 | Checkout with valid bKash       | +8801972742013, bKash                    | Order placed (90% success rate)              | Order placed                                 | PASS |
| TC-14 | Order more than stock           | Request 100 items, only 5 in stock       | Error: insufficient stock                    | InvalidQuantityException caught; error shown | PASS |
| TC-15 | View order history              | After placing order                      | Order appears in My Orders list              | Order visible in history                     | PASS |
| TC-16 | Update order status (admin)     | Select order, set DELIVERED              | Order status updated in database             | Status updated and confirmed                 | PASS |
| TC-17 | Delete product (admin)          | Select existing product                  | Product removed from catalogue               | Product deleted with confirmation            | PASS |
| TC-18 | Data persistence                | Register, add to cart, close app, reopen | Cart contents and user account preserved     | All data persisted                           | PASS |
| TC-19 | Update profile                  | Change email and address                 | Profile updated in database                  | Profile saved successfully                   | PASS |
| TC-20 | Wishlist add and move to cart   | Save product, move to cart               | Product removed from wishlist, added to cart | Wishlist and cart updated                    | PASS |

### 7.3 Edge Case Testing

After the normal flows were working, I deliberately tried inputs and situations that had caused problems while I was developing the application. The main edge cases I checked were:

- Empty form submission: Required fields are checked before submission. If a required value is missing, the form stays open and an error message is shown.

- Repeated cart additions: Adding the same product more than once increases its quantity instead of creating a second cart row. `CartService.addToCart()` handles this through its update logic.
- Zero-stock product: When stock reaches zero, the product is marked as out of stock and the Add to Cart action is disabled.
- Mobile number normalisation: The mobile-payment validation accepts both +8801972742013 and 01972742013 formats for bKash/Nagad numbers.
- First launch without a database: If `ecommerce.db` does not exist, `DatabaseSetup.createTables()` creates the required tables automatically using IF NOT EXISTS.
- Duplicate wishlist item: The same product cannot be saved twice for one customer. The UNIQUE constraint on `(customer_id, product_id)` prevents the duplicate and the interface shows an appropriate message.

## 7.4 Bugs Found & Fixed

| Bug ID | Description                                                   | Root Cause                                                                               | Fix Applied                                                                                                                 |
|--------|---------------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| BUG-01 | Admin login failed even with correct credentials              | <code>Admin.java</code> returned "Admin" but login checked for "ADMIN"                   | Changed Admin role string to "ADMIN" in both constructors and <code>getRole()</code>                                        |
| BUG-02 | <code>deleteProduct()</code> silently failed without deleting | SQL was "DELETE products WHERE id=?" missing FROM keyword                                | Corrected to "DELETE FROM products WHERE id=?"                                                                              |
| BUG-03 | <code>getTotalSales()</code> returned rounded integer values  | Result was stored in int variable instead of double                                      | Changed result variable type to double; used <code>rs.getDouble(1)</code>                                                   |
| BUG-04 | Duplicate products added on every app launch                  | <code>seedProducts()</code> in <code>Main.java</code> ran unconditionally                | Added <code>addProductIfMissing()</code> helper that checks before inserting                                                |
| BUG-05 | Cart items lost when user logged out and back in              | Cart was stored only in memory ( <code>Cart.java</code> )                                | Replaced in-memory cart with <code>CartService</code> using SQLite cart table                                               |
| BUG-06 | Dashboard did not open after login                            | <code>DashboardFrame</code> was using <code>DISPOSE_ON_CLOSE</code> which closed the app | Changed to <code>DISPOSE_ON_CLOSE</code> on all sub-frames; kept main <code>AppFrame</code> with <code>EXIT_ON_CLOSE</code> |

|        |                                       |                                                              |                                                                        |
|--------|---------------------------------------|--------------------------------------------------------------|------------------------------------------------------------------------|
| BUG-07 | Order placed even when payment failed | CheckoutFrame submitted order before checking payment result | processPayment() called first; order only submitted if it returns true |
|--------|---------------------------------------|--------------------------------------------------------------|------------------------------------------------------------------------|

## Chapter 8 Results & Discussions

### 8.1 System Demonstration Summary

By the end of the project, the main customer flow was working from start to finish. A visitor can open the storefront, browse or search for products and view their details. After creating an account and logging in, the same user can add products to the cart or wishlist, go through checkout, update profile information and view previous orders.

I also implemented the administrator side separately from the customer flow. After an admin logs in, the dashboard provides product management, image upload, customer management, order-status controls and reports. I kept these options away from customer accounts by checking the logged-in user's role.

The application's data remains available after it is closed because it is stored in SQLite. The program is designed to run locally without an internet connection and does not require a separate database server.

### 8.2 Requirement Fulfill Metrix

| Requirement ID | Feature                               | Status |
|----------------|---------------------------------------|--------|
| FR-01          | User registration                     | Met    |
| FR-02          | Role-based login                      | Met    |
| FR-03          | SHA-256 password hashing              | Met    |
| FR-04          | Duplicate username detection          | Met    |
| FR-05          | Profile update                        | Met    |
| FR-06          | Product CRUD with image               | Met    |
| FR-07          | Product browse with search and filter | Met    |
| FR-08          | Sort products                         | Met    |
| FR-09          | Product details dialog                | Met    |

|       |                                 |     |
|-------|---------------------------------|-----|
| FR-10 | Add to cart with quantity       | Met |
| FR-11 | Cart management                 | Met |
| FR-12 | Wishlist                        | Met |
| FR-13 | Checkout with 3 payment methods | Met |
| FR-14 | Payment validation              | Met |
| FR-15 | Order history                   | Met |
| FR-16 | Order details                   | Met |
| FR-17 | Admin order management          | Met |
| FR-18 | Customer management             | Met |
| FR-19 | Summary statistics              | Met |
| FR-20 | Reports and analytics           | Met |

### 8.3 OOP Concepts Coverage

| OOP Concept                  | Implementation                                                                                       | Classes/Methods                                                         | Status |
|------------------------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|--------|
| Encapsulation                | Private fields, getters/setters, SHA-256 hashing                                                     | All model classes                                                       | Met    |
| Inheritance                  | Two-level hierarchy for User and Payment                                                             | User→Customer/Admin;<br>Payment→CashPayment/MobilePayment/CreditCardPay | Met    |
| Abstraction (Abstract Class) | Abstract User and Payment classes with abstract methods                                              | User.getRole();<br>Payment.processPayment()                             | Met    |
| Abstraction (Interface)      | Payable interface implemented by Payment                                                             | Payable → Payment → all payment types                                   | Met    |
| Polymorphism (Overriding)    | processPayment() overridden in all three payment classes; getRole() overridden in Customer and Admin | CheckoutFrame uses parent reference; runtime dispatch                   | Met    |
| Polymorphism (Overloading)   | addProduct(), updateProduct(), searchProduct() each have 2 overloaded versions                       | ProductService                                                          | Met    |
| Collections Framework        | ArrayList for all product/order/cart lists; LinkedHashSet for categories                             | Throughout all service and UI classes                                   | Met    |

|                     |                                                                                          |                                                |     |
|---------------------|------------------------------------------------------------------------------------------|------------------------------------------------|-----|
| Exception Handling  | 4 custom exceptions; try-catch throughout; transaction rollback on order failure         | exception package; OrderService; CheckoutFrame | Met |
| File I/O / Database | SQLite persistence via JDBC; file I/O for product images                                 | All service classes; ImageUtil                 | Met |
| Static Members      | All service methods static; DBConnection.connection static; Theme constants static final | All service classes; DBConnection; Theme       | Met |

## 8.4 Limitations

- Payment simulation: The current payment process is only a simulation; it is not connected to a real payment gateway. Card and mobile payments use a simulated success result, so they should not be treated as real bank responses.
- Single-user session: Only one user can be logged in to the desktop application at a time. The system does not support multiple simultaneous sessions.
- No email notifications: Features such as registration emails, order-confirmation emails, and password-reset emails are not included in the current version.
- Static service methods: The service layer mostly uses static methods. This keeps the project simpler, but it would make more advanced unit testing with mocks harder.
- No product reviews: Customers can browse and buy products, but they cannot currently leave ratings or written reviews.
- Image storage: The database stores the file path of each product image rather than the image itself. Because of this, moving or reorganizing the application files could break saved image paths.

## **Chapter 9 Conclusions & Future Work**

### **9.1 Conclusion**

Overall, the project helped me use encapsulation, inheritance, abstraction and polymorphism in something larger than the short examples normally used when learning OOP. Some concepts fitted naturally in particular areas, especially inheritance for users and polymorphism for payments, while encapsulation is used across most of the model classes.

SQLite gives the application persistent storage without requiring a separate database server, which keeps the setup simple. Order placement also uses database transactions so related changes can be completed together or rolled back if something fails during checkout.

Working on the project also exposed me to parts of Java that I had not used much before, especially Swing, JDBC and database transactions. The biggest difference from smaller exercises was having to make all of these parts work together and then fix problems when a change in one area affected another.

### **9.2 Individual Reflection**

When I started the project, Java GUI programming was one of the areas I was least familiar with. At first I spent a lot of time understanding Swing layouts and how to move between screens. As the project grew, I also had to learn not to put every operation directly inside button listeners and instead pass work to the service and model classes.

The checkout process was one of the hardest parts to organize. Payment validation, order creation, order-item insertion, stock reduction, and cart clearing all depend on each other. Implementing them as one transaction helped me understand why database consistency matters and why a failed step should not leave half of an order saved.

The wishlist feature was a good example of why this separation helped. I could add most of it through WishlistService and WishlistFrame without rewriting the product or order logic. Seeing that work in my own code made the reason for modular design much easier to understand than it was from theory alone.

### 9.3 Future Enhancements

1. Real payment gateway integration: A future version could connect to the bKash Merchant API or another sandbox-supported gateway so payment results are verified by an actual service instead of being simulated.
2. Email notifications: Registration confirmations, order updates, and other useful messages could be sent automatically through an email service such as JavaMail.
3. Product ratings and reviews: Customers who have purchased a product could be allowed to give a 1-5 star rating and leave a short review, with the average rating displayed on the storefront.
4. Discounts and coupons: An admin could create coupon codes with percentage or fixed-value discounts, expiry dates, and simple usage rules.
5. Inventory alerts: Instead of requiring the admin to open the reports page, the system could show an in-app warning when a product falls below a chosen stock level.
6. DAO layer refactoring: Database queries could be moved into dedicated classes such as UserDAO, ProductDAO, and OrderDAO. This would separate SQL more clearly from business logic and make the services easier to test.
7. Automated unit testing: JUnit tests could be added for the service layer, using a temporary or in-memory SQLite database so database operations can be tested without changing normal application data.



## References

- [1] B. Eckel, Thinking in Java, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2006.
  
- [2] Oracle Corporation, "The Java Tutorials Learning the Java Language," Oracle, 2024. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>. [Accessed: July 2026].
  
- [3] Oracle Corporation, "Java SE 26 API Documentation," Oracle, 2026. [Online]. Available: <https://docs.oracle.com/en/java/javase/26/docs/api/>. [Accessed: July 2026].
  
- [4] SQLite Consortium, "SQLite Documentation," SQLite, 2024. [Online]. Available: <https://www.sqlite.org/docs.html>. [Accessed: July 2026].
  
- [5] xerial, "sqlite-jdbc: SQLite JDBC Driver," GitHub, 2024. [Online]. Available: <https://github.com/xerial/sqlite-jdbc>. [Accessed: July 2026].
  
- [6] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.
  
- [7] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River, NJ: Prentice Hall, 2008.

## Appendices

### Appendix A Source Code

The complete source code is available on GitHub at:

<https://github.com/fahady724/eccommerce>

The repository contains:

- src/ All Java source files organised in model, service, ui, exception, util packages
- sqlite-jdbc-3.53.2.0.jar SQLite JDBC driver
- README.md Setup and run instructions
- .gitignore Excludes compiled files and database from version control

### Appendix C Team Contribution

| Member              | Student ID | Tasks                                                       | Contribution |
|---------------------|------------|-------------------------------------------------------------|--------------|
| Fahad Fardin Younus | 253174038  | All design, implementation, testing, and 100% documentation |              |

### Appendix D Development Log

| Date      | Activity                                        | Outcome                                                         |
|-----------|-------------------------------------------------|-----------------------------------------------------------------|
| June 2026 | Project planning and technology selection       | Chose Java Swing + SQLite; designed class hierarchy             |
| June 2026 | Model classes and exception classes implemented | User, Product, Order, Payment hierarchy complete                |
| June 2026 | Database setup and service layer implemented    | AuthService, ProductService, OrderService, CartService complete |
| June 2026 | WishlistService and ReportService implemented   | Analytics and wishlist features complete                        |
| July 2026 | GUI implementation storefront, login, register  | Public-facing screens complete and functional                   |
| July 2026 | GUI implementation admin dashboard, checkout    | Admin features and checkout flow complete                       |
| July 2026 | Bug fixing and testing                          | All 7 known bugs identified and fixed                           |
| July 2026 | Report writing and GitHub cleanup               | Report complete; repository organised                           |